So ends the scenic route through binding. From here, you move on to the foundation of CoffeeScript objects: prototypes.

4.6. Prototypes

CoffeeScript is a prototype-based language. Every object (except `null`—you'll learn why later) has a link to another object called its *prototype*. Properties defined on the prototype of an object are also available on the object itself. A prototype-based object system will be unfamiliar to those who're used to the class-based object systems found in most popular programming languages because it supports object creation and inheritance without requiring any classes at all.

In this section you'll learn what it means to create a new object using an existing object as the prototype and how an object inherits properties from its prototype. To understand this, it's important to distinguish inheritance from copying.

4.6.1. Copy

Back when people wore leg warmers, fluorescent clothes, and their hair in a perm, music was distributed on small plastic cartridges called cassettes. Imagine your friend Corey has lent you a cassette he made that contains some of his favorite songs:

```
cassette =
  title: "Awesome songs. To the max!"
  duration: "10:34"
  released: "1988"
  track1: "Safety Dance - Men Without Hats"
  track2: "Funkytown - Lipps, Inc"
  track3: "Electric Avenue - Eddy Grant"
  track4: "We Built This City - Starship"
```

Making a copy

You really like the cassette and decide to create a copy of it: